School of Electrical and Computer Engineering,
Technical University of Crete
Academic Υear 2023-2024 (Spring Semester)
INF424 Functional Programming, Analytics and Applications

Instructor: N. Giatrakos, Lab Tutor: C. Tsinaraki

Project Topic: Functional Programming Analytics and Applications with Scala on Apache Spark

Author : Panagiotis Georgakopoulos - Paltidis
AM: 2015030049

Preamble on Implementation Specifications

In the scope of this project, we are going to use functional programming to perform Big Data analytics over a state-of-the-art Big Data Platform, namely Apache Spark.

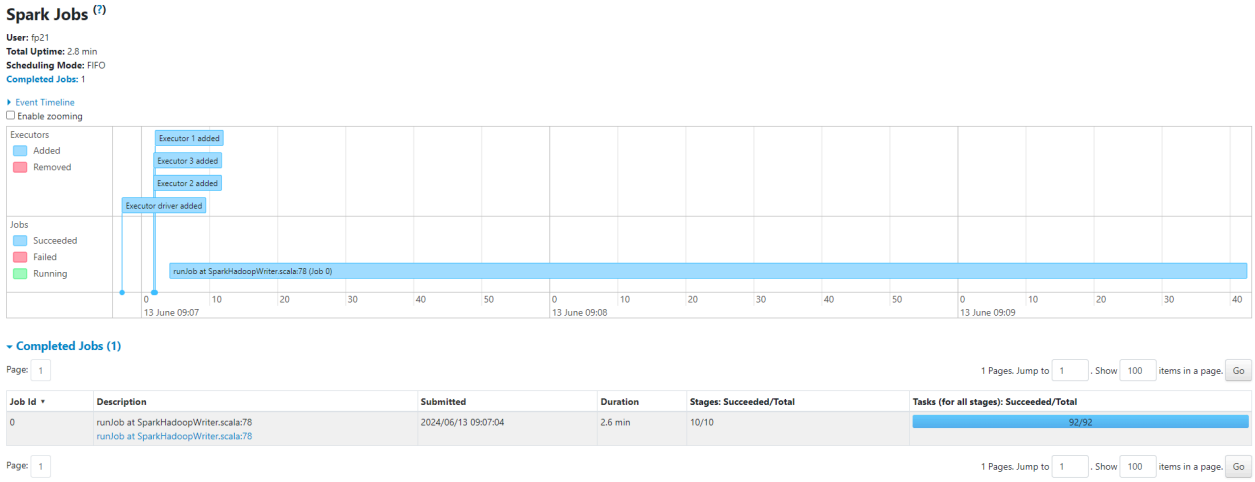
Application Scenario 1: Reuters News Stories Analysis using Spark RDDs (3 Units)

In the context of this application scenario, we will use the files available in the Reuters Dataset. This is a semi-structured dataset, therefore one has to use Spark RDDs to proceed with the analysis.This dataset includes a set of files with information about **news stories (documents)** which are assigned to specific **categories** each. Each document also includes a set of **terms** of interest to analyze in conjunction with existing news categories.

More specifically, the purpose of the analysis is, by compiling the information in the various records, to extract the relevance of each term to each category. To quantify this degree of relevance, we will use the Jaccard Index as a measure.

The following applications were developed and tested locally at first and then modified to run on the TUC SoftNet Cluster by making the necessary changes to the sbt build , the dfs File Paths and finally configuring the SparkSession Builder to have Yarn as a Master and the correct endpoints.

The following images were extracted from the event logs produced by the history server running on the TUC SoftNet Cluster.



Apache Spark is a unified analytics engine for large-scale data processing improving on Hadoop by utilizing its in-memory cluster computing and thus reducing the amount of read/writes on HDFS during the shuffling process. YARN follows the Master/Slave Architecture where the driver does all the scheduling and stores the metadata info on all the datanodes while the executors is where all the data processing is performed.The workflow begins by executing the spark context on the driver and then 3 executors are added. The amount of workers was specified in the spark-submit command along with other specifications such as CPU Memory and more.As we can see in the image above there is only a single job executed for the entirety of the program.That is because jobs are decided by spark based on actions and there was only a single action in the code namely line 85 as shown below.

```
84    hdfs.delete(new Path(outputPath), true)
85    finalResult.saveAsTextFile(outputPath)
```

Everything else is narrow and wide transformations performed on RDDS with 10 stages and 92 tasks succeeded in total. The DAG Scheduler divides the execution plan into stages where each stage is a set of transformations that can be executed without requiring a shuffle (usually caused by Wide transformations) and each task corresponds to a partition of the data. In our case the maximum achieved level of parallelism should be 3 because we only have 3 executors. We will confirm this later down the line when we examine the details for each stage.

The analysis begins by reading the 5 ASCII files containing the Reuters-assigned document ID and the term ID into a single RDD named termFileRDD and then creating 2 new RDDs DOC_CAT_COUNT and DOC_TERM_COUNT where we store a number showing how many Document IDS exist for each category and term respectively.Below is the corresponding lines of code.

```scala
val termFileRDD = sc.textFile(termFilePath1)
  .union(sc.textFile(termFilePath2))
  .union(sc.textFile(termFilePath3))
  .union(sc.textFile(termFilePath4))
  .union(sc.textFile(termFilePath5))

val DOC_CAT_COUNT = sc.textFile(categoryFilePath).map(line => (line.split("\\s+") // 1. count DOCID for each category
    .headOption
    .getOrElse().toString , 1))
  .reduceByKey(_ + _)

val DOC_TERM_COUNT = termFileRDD.flatMap(_.split(":")) // 2. count DOCID for each termID
  .flatMap(_.split("\\s+").tail)
  .map(term => (term , 1) )
  .reduceByKey(_ + _)
```

After some data cleaning and performing a join on DOC ID and reducing by key

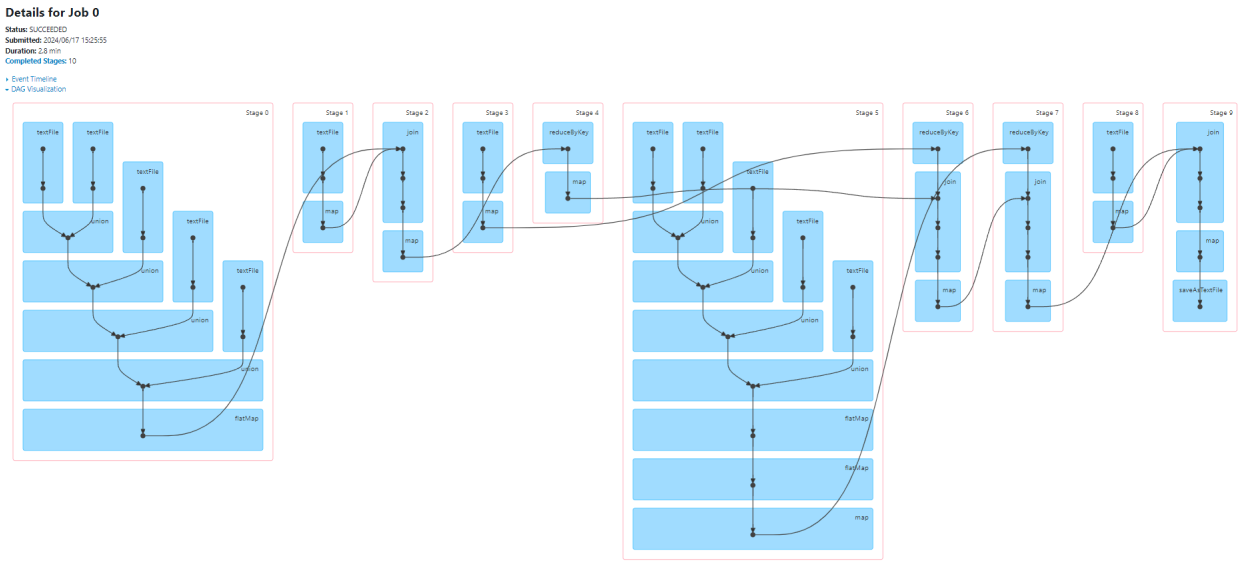
| DOC (T) ∩ DOC (C) | is calculated as shown in the lines below.

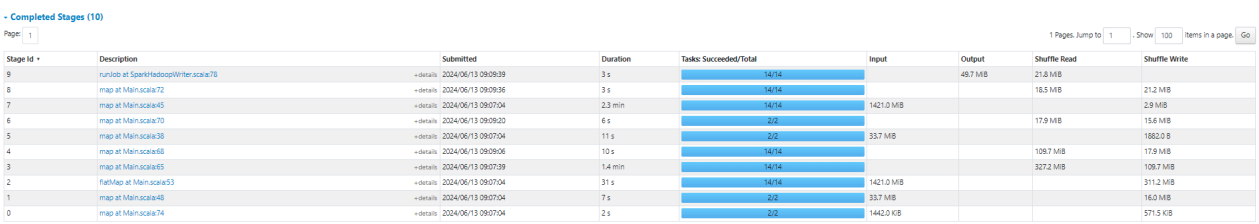
```scala
val DOC_C = sc.textFile(categoryFilePath).map(line => (line.split("\\s+").tail.headOption.getOrElse().toString ,
  line.split("\\s+")
    .headOption
    .getOrElse().toString))          // drop the 1 and transform to (DOCID , Category)

val DOC_T = termFileRDD.flatMap { line =>
  val words = line.split(" ")
  val docID = words(0)              // extract docID
  val DocTermPair = words.drop(1)   // Drop docID

  DocTermPair.map { t =>           // map to (docID, termID) filter if termID empty
    val termID = t.split(":")(0)
    (docID, termID)
  }.filterNot(pair => pair._2.isEmpty)
}

val DOCintersectionCount = DOC_C.join(DOC_T)  // join on docID and then reduce by key => ((Category,termID),count)
  .map( x => (x._2,1) )
  .reduceByKey(_ + _)
```

Finally by joining and mapping the 3 RDDs DocIntersectionCount , DOC_CAT_COUNT and DOC_TERM_COUNT the jaccardIndex is calculated.Then the stem ID is extracted and the final result is produced.

```scala
val jaccardIndex = DOCintersectionCount.map( x => (x._1._1,(x._1._2,x._2)))                  : RDD[(String, (String, Int))]
  .join(DOC_CAT_COUNT)                                                                        : RDD[(String, ((String, Int), Int))]
  .map( x => (x._2._1._1,(x._1,x._2._1._2,x._2._2)))                                          : RDD[(String, (String, Int, Int))]
  .join(DOC_TERM_COUNT)                                                                       : RDD[(String, ((String, Int, Int), Int))]
  .map( x => (x._1,(x._2._1._1,x._2._1._2.toDouble / (x._2._1._3 + x._2._2 - x._2._1._2)))) : RDD[(String, (String, Double))]
// calculating Jaccard Index
// 1. make category key
// 2. join on category with DOC_CAT_COUNT to get the count
// 3. make termID key
// 4. join to get termID count
// 5. calculate Jaccard Index map to (termID,(docID,jaccardIndex))

val finalResult = sc.textFile(stemFilePath).map(line =>  : RDD[String]
    (line.split("\\s+")(1) , line.split("\\s+")(0)))     : RDD[(String, String)]
  .join(jaccardIndex)                                    : RDD[(String, (String, (String, Double)))]
  .map( x => (x._2._2._1,x._2._1,x._2._2._2))            : RDD[(String, String, Double)]
// Get the final result in requested format
// 1. read stem file and map to (TermID,stem) in order to join with jaccard Index RDD
// 2. join with jaccard Index RDD
// 3. map to final format (category,stem,JaccardIndex) and drop the termID

hdfs.delete(new Path(outputPath), true)
finalResult.saveAsTextFile(outputPath)
```

Below is the Dag Visualization showing the stages and a table with the tasks completed for each stage.





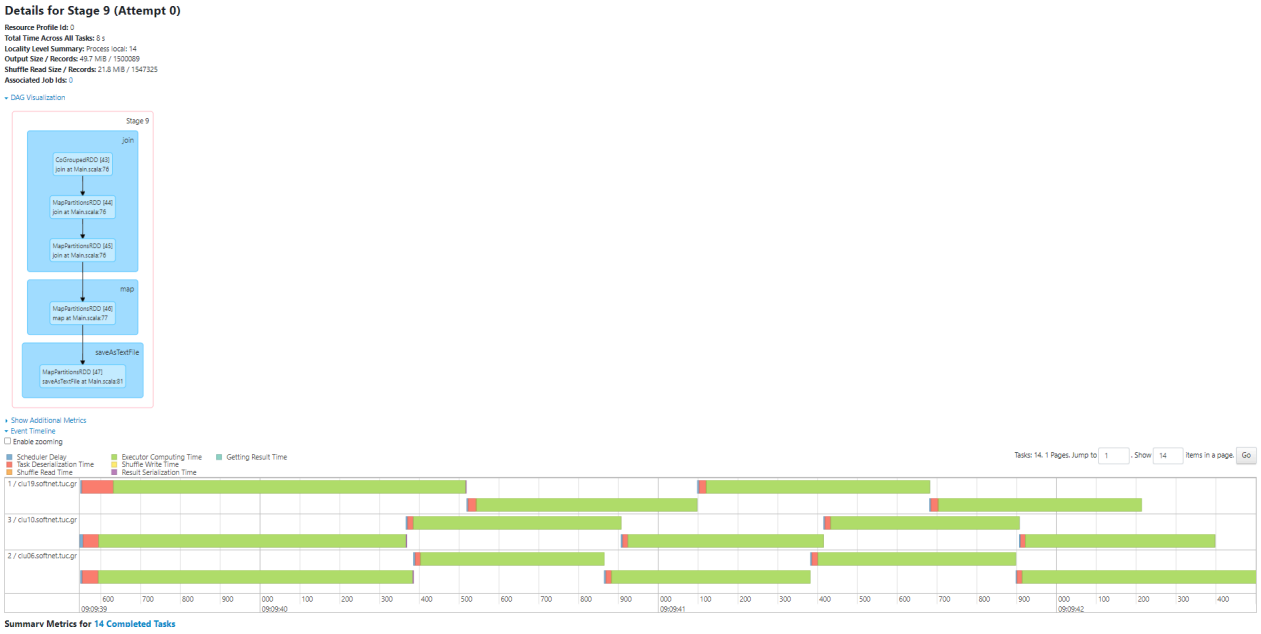
Completed Stages (10)

Page: 1

1 Pages. Jump to 1 . Show 100 Items in a page. Go

Stage Id	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
9	runJob at SparkHadoopWriter.scala:78	+details	2024/06/13 09:09:39	3 s	14/14		49.7 MiB	21.8 MiB	
8	map at Main.scala:72	+details	2024/06/13 09:09:36	3 s	14/14			18.5 MiB	21.2 MiB
7	map at Main.scala:45	+details	2024/06/13 09:07:04	2.3 min	14/14	1421.0 MiB			2.9 MiB
6	map at Main.scala:70	+details	2024/06/13 09:09:20	6 s	2/2			17.9 MiB	15.6 MiB
5	map at Main.scala:38	+details	2024/06/13 09:07:04	11 s	2/2	33.7 MiB			1882.0 B
4	map at Main.scala:68	+details	2024/06/13 09:09:06	10 s	14/14			109.7 MiB	17.9 MiB
3	map at Main.scala:65	+details	2024/06/13 09:07:39	1.4 min	14/14			327.2 MiB	109.7 MiB
2	flatMap at Main.scala:53	+details	2024/06/13 09:07:04	31 s	14/14	1421.0 MiB			311.2 MiB
1	map at Main.scala:48	+details	2024/06/13 09:07:04	7 s	2/2	33.7 MiB			16.0 MiB
0	map at Main.scala:74	+details	2024/06/13 09:07:04	2 s	2/2	1442.0 KiB			571.5 KiB

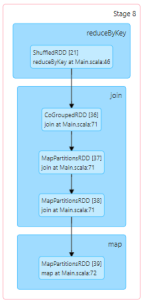
The following are 3 stages showing that the maximum achieved parallelism is 3 as expected. We can also see that the first operation of every stage is a wide operation requiring a shuffle in the DAG Visualization.



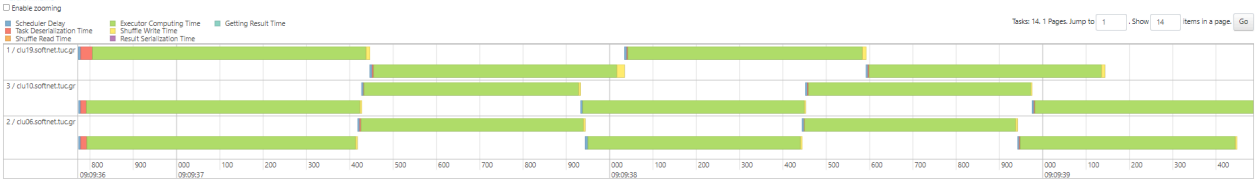
Details for Stage 8 (Attempt 0)

Resource Profile Id: 0
Total Time Across All Tasks: 8 s
Locality Level Summary: Node local: 14
Shuffle Read Size / Records: 18.5 MiB / 1961242
Shuffle Write Size / Records: 21.2 MiB / 1500089
Associated Job Ids: 0

DAG Visualization



Show Additional Metrics
Event Timeline
Enable zooming

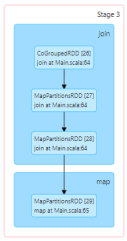


Summary Metrics for 14 Completed Tasks

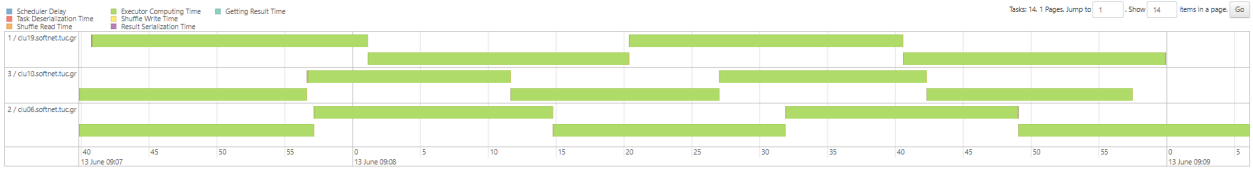
Details for Stage 3 (Attempt 0)

Resource Profile Id: 0
Total Time Across All Tasks: 4.1 min
Locality Level Summary: Process local: 14
Shuffle Read Size / Records: 327.2 MiB / 63521988
Shuffle Write Size / Records: 109.7 MiB / 9459087
Associated Job Ids: 0

DAG Visualization



Show Additional Metrics
Event Timeline
Enable zooming



Summary Metrics for 14 Completed Tasks

Application Scenario 2:
Vessel Trajectory Analytics in the Aegean Sea using Spark Dataframes (20%)

In this application scenario we are going to perform basic analysis on vessel position tracking data collected from a set of stations located in Piraeus and Syros Island.

Q1: What is the number of tracked vessel positions per station per day?

Q2: What is the vessel id with the highest number of tracked positions?

Q3: What is the average SOG of vessels that appear in both station 8006 and station 10003 in the same day.

Q4: What is the average Abs (Heading – COG) per station?

Q5: What are the Top-3 most frequent vessel statuses?

Although our data was given in a txt file the format of the data allowed us to conveniently read our data into a Dataframe using the csv read function and the header option as true without the need to do any serious processing.After that we did the only data cleaning required by isolating the date from the timestamp.

```
23    val newNames = Seq("date", "station", "vesselID", "longitude", "latitude" , "SOG" , "COG" , "heading" , "status")
24
25    val df = spark.read.option("header" , "true").csv(input).toDF(newNames: _*) // read the txt as a dataframe using read.csv and dropping the header
26
27    val filteredDF = df.withColumn("date" , col("date").substr(0,10)) // isolate the date from timestamp
```

12 jobs were completed in total as shown below. 10 of them were because we were asked to both save the results in an output file in HDFS and print them in the driver 1 more is the read.csv and the final one is in line 48 where .first() is used in order to print the first result with some text.

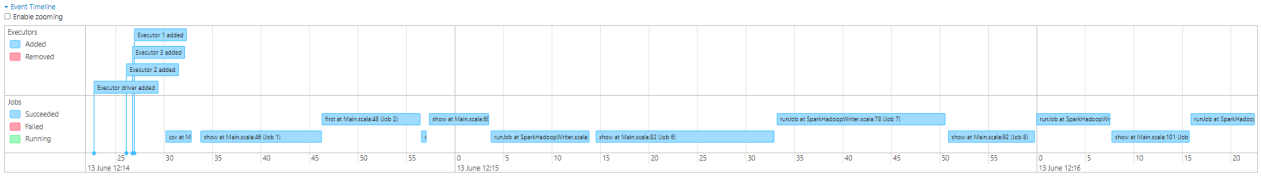
```
48    println("Number of tracked vessel positions per station per day: " + TrackedPositionsRDD.first().mkString(""))
```

Below is a table showing all of the jobs and the event timeline.

User: fp21
Total Uptime: 2.0 min
Scheduling Mode: FIFO
Completed Jobs: 12

Event Timeline

Enable zooming



Completed Jobs (12)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
11	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/06/13 12:16:15	7 s	3/3	204/204
10	show at Main.scala:101 show at Main.scala:101	2024/06/13 12:16:07	8 s	2/2	203/203
9	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/06/13 12:15:59	8 s	3/3	204/204
8	show at Main.scala:92 show at Main.scala:92	2024/06/13 12:15:50	9 s	3/3	204/204
7	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/06/13 12:15:33	17 s	6/6	607/607
6	show at Main.scala:82 show at Main.scala:82	2024/06/13 12:15:14	18 s	6/6	607/607
5	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/06/13 12:15:03	10 s	3/3	204/204
4	show at Main.scala:60 show at Main.scala:60	2024/06/13 12:14:57	6 s	2/2	203/203
3	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/06/13 12:14:56	0.5 s	1/1 (3 skipped)	1/1 (403 skipped)
2	first at Main.scala:48 first at Main.scala:48	2024/06/13 12:14:46	10 s	4/4	404/404
1	show at Main.scala:46 show at Main.scala:46	2024/06/13 12:14:33	12 s	4/4	404/404
0	csv at Main.scala:31 csv at Main.scala:31	2024/06/13 12:14:30	3 s	1/1	1/1

Page: 1

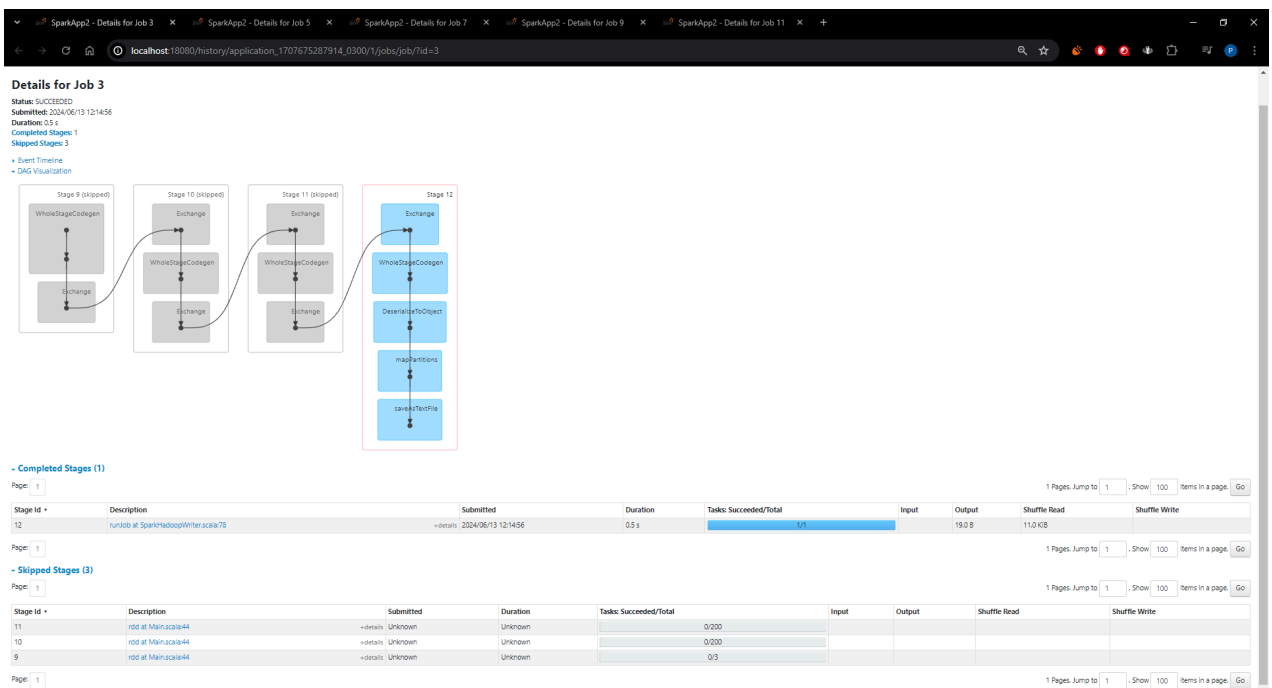
1 Pages. Jump to 1 . Show 100 items in a page. Go

Below is the code and corresponding job for each saveAsTextFile

Code for Q1:

```scala
val TrackedPositions = filteredDF.groupBy(
    col("date") , col("station")
  ).count()
  .groupBy(
    col("station")
  ).avg()                                                    // get the avg tracked vessel positions for each station per day

val trackedPositionsAvg = TrackedPositions.groupBy().avg()        // get final result

val TrackedPositionsRDD = trackedPositionsAvg.rdd

trackedPositionsAvg.show()

println("Number of tracked vessel positions per station per day: " + TrackedPositionsRDD.first().mkString(""))

hdfs.delete(new Path(outputPath1), true)
TrackedPositionsRDD.saveAsTextFile(outputPath1)
```

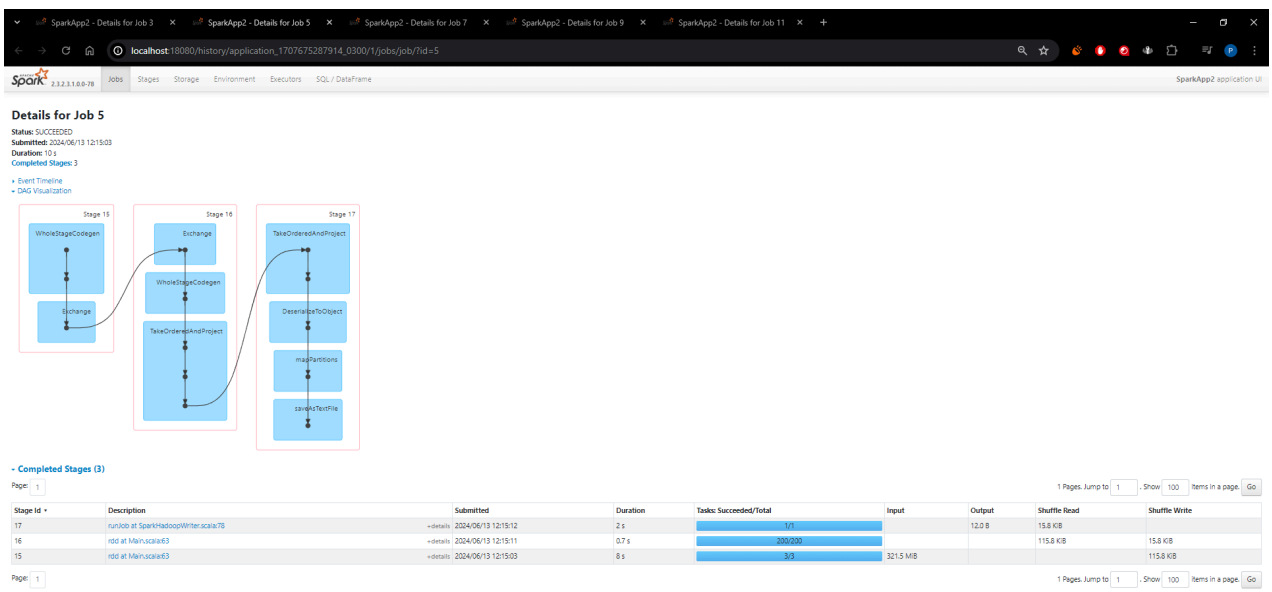
Job for Q1:



Code for Q2:

```
val maxVesselID = filteredDF.groupBy(
    col("vesselID").as("VesselID with max positions tracked")
  ).agg(count(col("VesselID")).as("Positions Tracked"))
  .orderBy(col("Positions Tracked").desc).limit(1)

val q2 = maxVesselID.select("VesselID with max positions tracked")

q2.show()

hdfs.delete(new Path(outputPath2), true)
q2.rdd.saveAsTextFile(outputPath2)
```

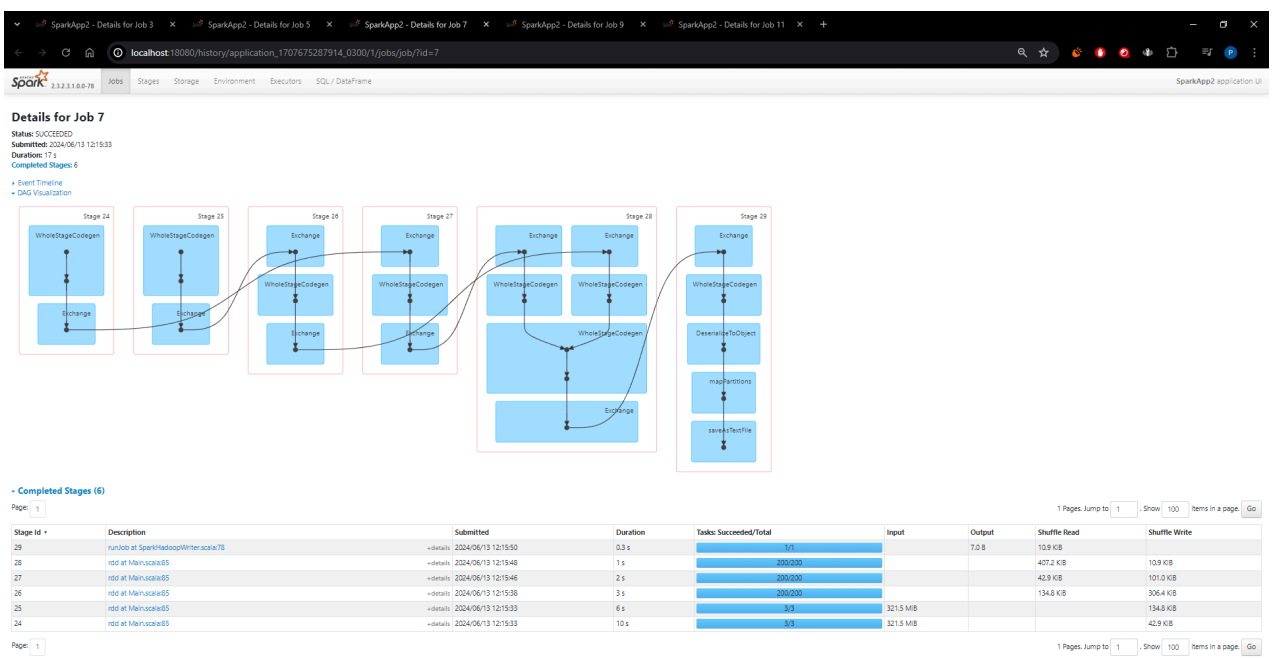
Job for Q2:



Code for Q3:

```scala
val temp1 = filteredDF.groupBy(
  col("vesselID") , col("date") , col("station")
).agg(
  avg(col("SOG")).as("SOG")
).filter(col("station") === 8006)

val names2 = Seq("vesselID2", "date2", "station2", "SOG2")

val temp2 = filteredDF.groupBy(
  col("vesselID") , col("date") , col("station")
).agg(
  avg(col("SOG")).as("SOG")
).filter(col("station") === 10003).toDF(names2: _*)

val q3 = temp1.join(temp2 , temp1("vesselID") === temp2("vesselID2") && temp1("date") === temp2("date2"))
  .agg(avg((col("SOG") + col("SOG2"))/2).as("Average SOG"))

q3.show()

hdfs.delete(new Path(outputPath3), true)
q3.rdd.saveAsTextFile(outputPath3)
```

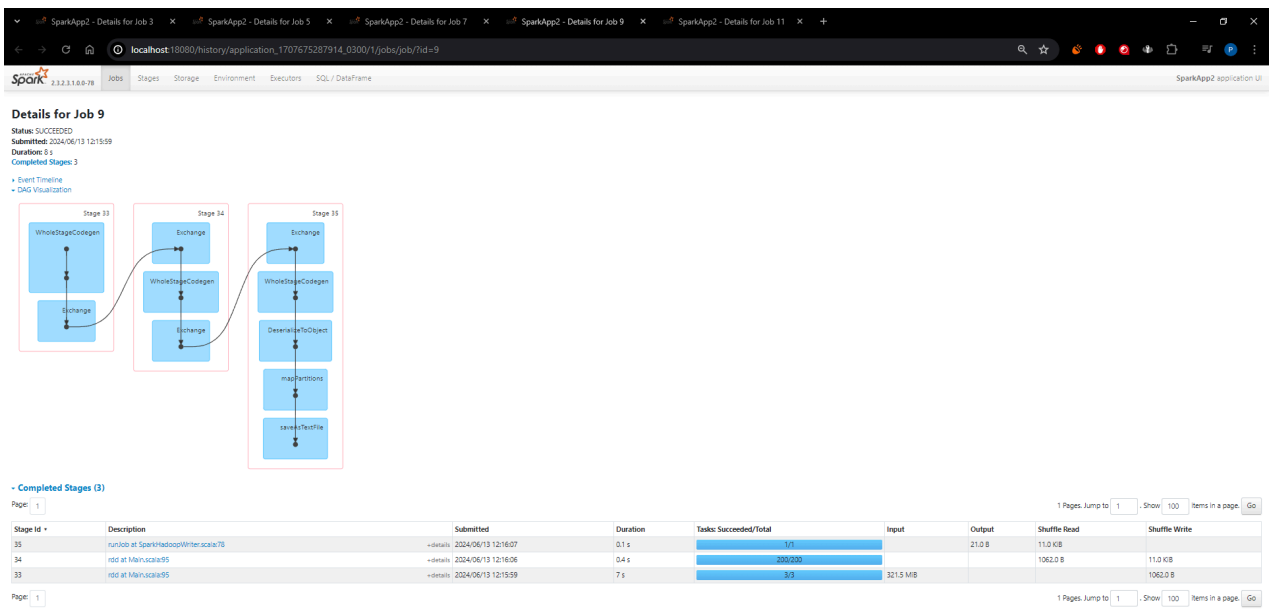
Job for Q3:



Code for Q4:

```
val q4 = filteredDF.groupBy( col("station") )
  .agg(
    avg(abs(col("heading") - col("COG"))).as("avg")
  ).groupBy().agg(avg(col("avg")).as("Average Abs(Heading - COG) per station"))

q4.show()

hdfs.delete(new Path(outputPath4), true)
q4.rdd.saveAsTextFile(outputPath4)
```

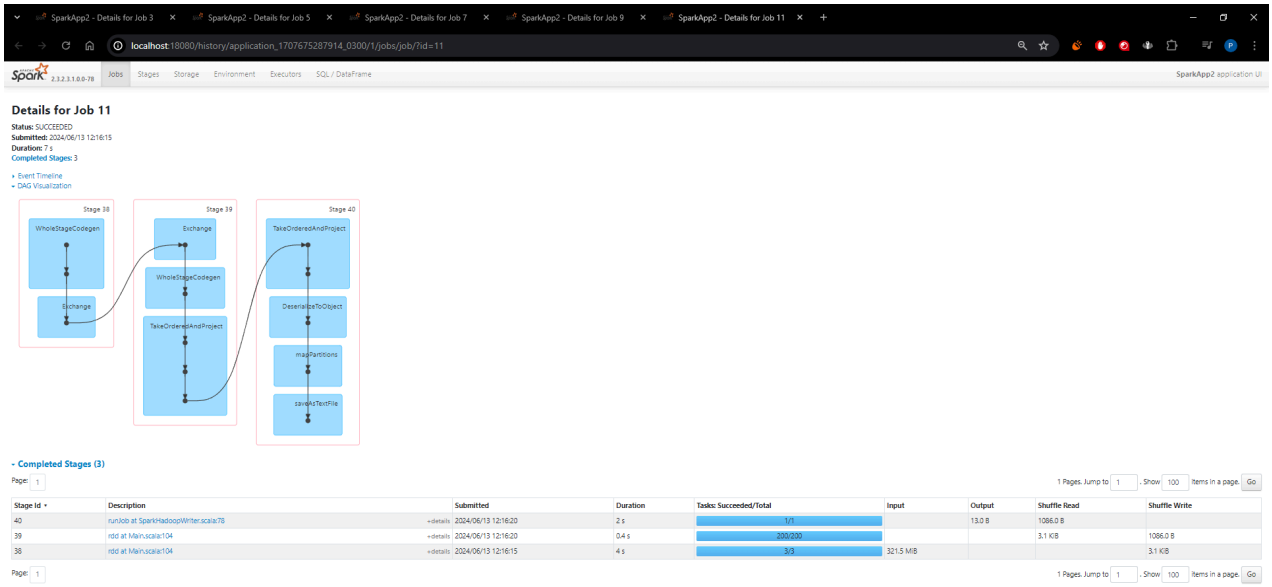
Job for Q1:



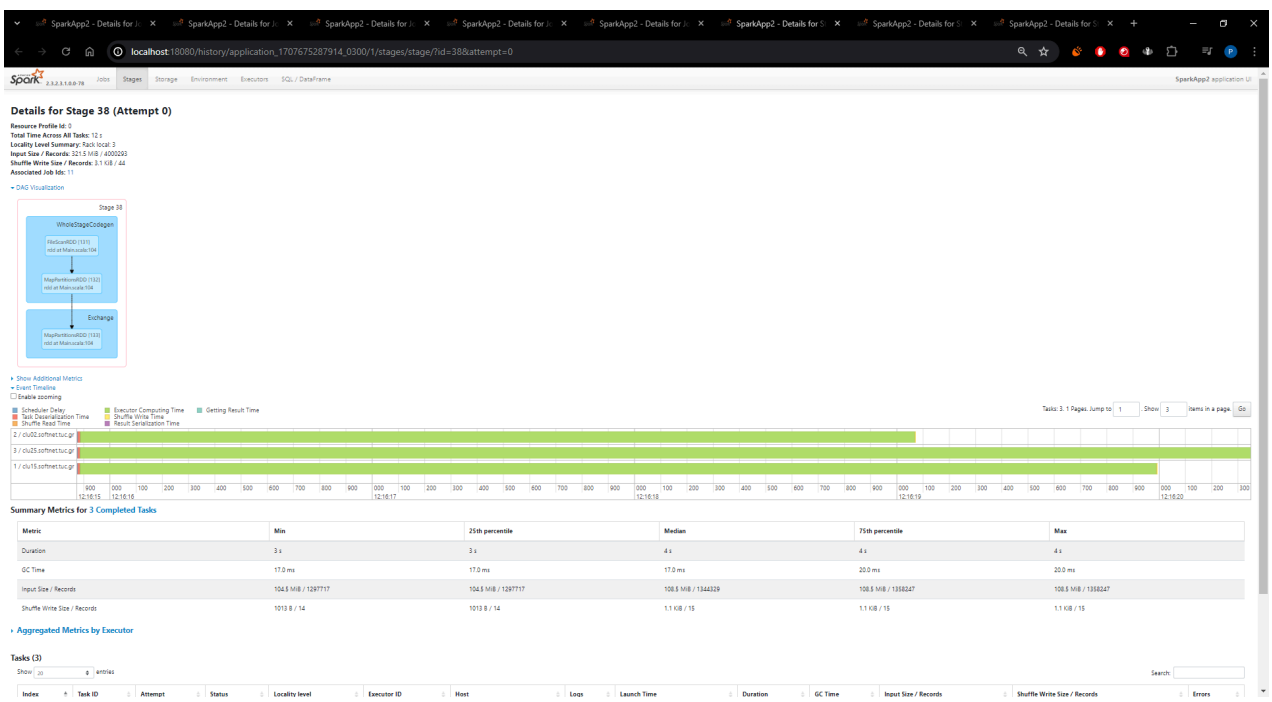
Code for Q5:

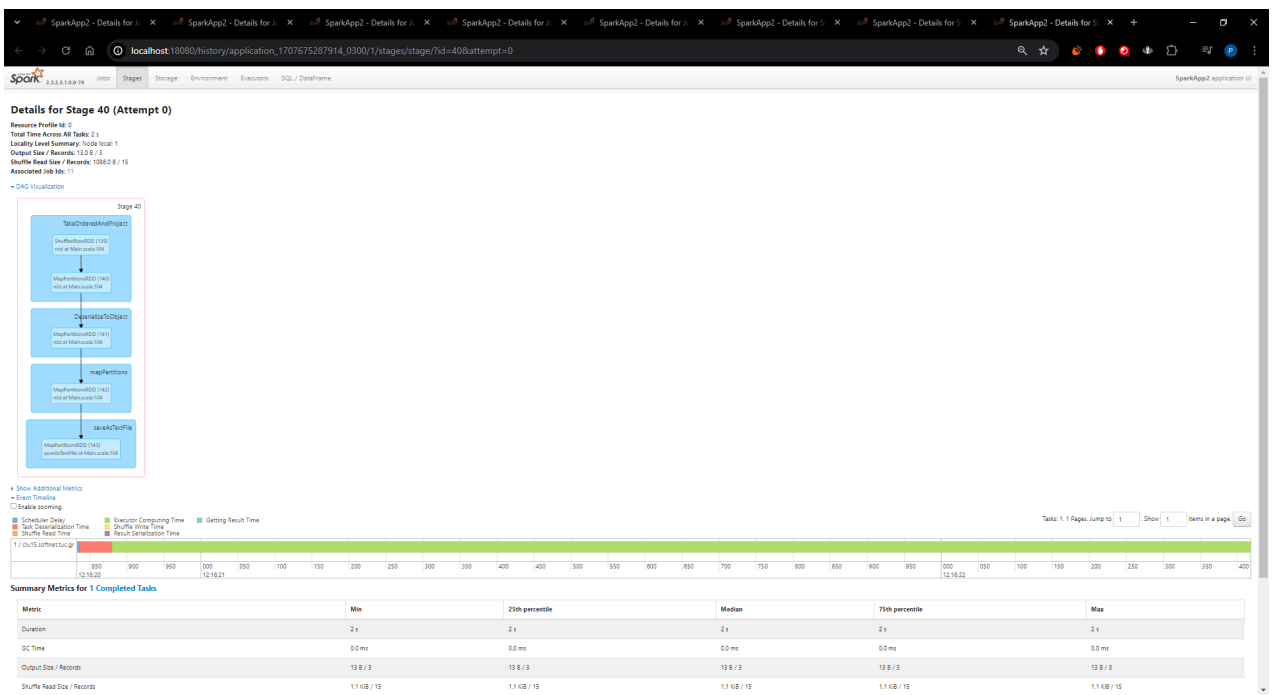
```
val q5 = filteredDF.groupBy(col("status"))
  .agg( count(col("status")).as("count"))
  .orderBy(col("count").desc).select(col("status").as("Top-3 Frequent Statuses")).limit(3)

q5.show()

hdfs.delete(new Path(outputPath5), true)
q5.rdd.saveAsTextFile(outputPath5)
```

Job for Q5:



Indicative stage details for job 11:





Once again we can see the maximum achieved level of parallelism is 3 even though the number of tasks for each stage has increased.